Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Программирование мобильных приложений»

ОТЧЕТ
к лабораторной работе №4
на тему:
«ОНЛАЙН-ИГРА "МОРСКОЙ БОЙ"»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
БУРАКОВ Дмитрий Игоревич

_____
(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

1 Два игрока должны иметь возможность участвовать в игре с разных устройств.

2 Firebase Firestore / RealtimeDB или аналогичный сервис используется в качестве бэкенда.

3 Аутентификация с использованием любого провайдера или простой пары email-пароль.

4 Пользователь 1 создает игру и делится сгенерированным ID игры с противником.

5 Противник (Пользователь 2) присоединяется к игре по полученному ID.

6 Устройство Пользователя 1 выступает в роли сервера и отвечает за обновление состояния игры.

7 Устройство Пользователя 2 подписывается на изменения состояния в базе данных и обновляет интерфейс.

8 Страница статистики пользователя с предыдущими играми.

9 Страница профиля пользователя: изменение никнейма; аватар; удаленное хранилище.

10 Корректная обработка отказа в доступе к разрешениям Android.

11 В отчете по лабораторной работе представить следующие диаграммы: диаграмму последовательности, диаграмму развертывания, диаграмму состояний.

2 ВЫПОЛНЕНИЕ РАБОТЫ

В ходе выполнения лабораторной работы было разработано мобильное приложение "Морской бой" для платформы Android. Приложение реализовано на языке Kotlin с использованием Jetpack Compose для построения пользовательского интерфейса, Navigation Compose для маршрутизации между экранами. В качестве облачной инфраструктуры используются Firebase Authentication и Cloud Firestore, что позволяет выполнять авторизацию пользователей, хранить профили игроков и синхронизировать состояние партии между двумя устройствами в режиме, близком к реальному времени.

Стартовой точкой приложения является экран авторизации, на котором пользователь может выполнить вход по email и паролю либо зарегистрировать новый аккаунт. При регистрации формируется документ пользователя в коллекции users, содержащий идентификатор, никнейм, выбранный аватар и счетчики игровых показателей. Экран авторизации приложения рекомендуется разместить на рисунке 2.1.

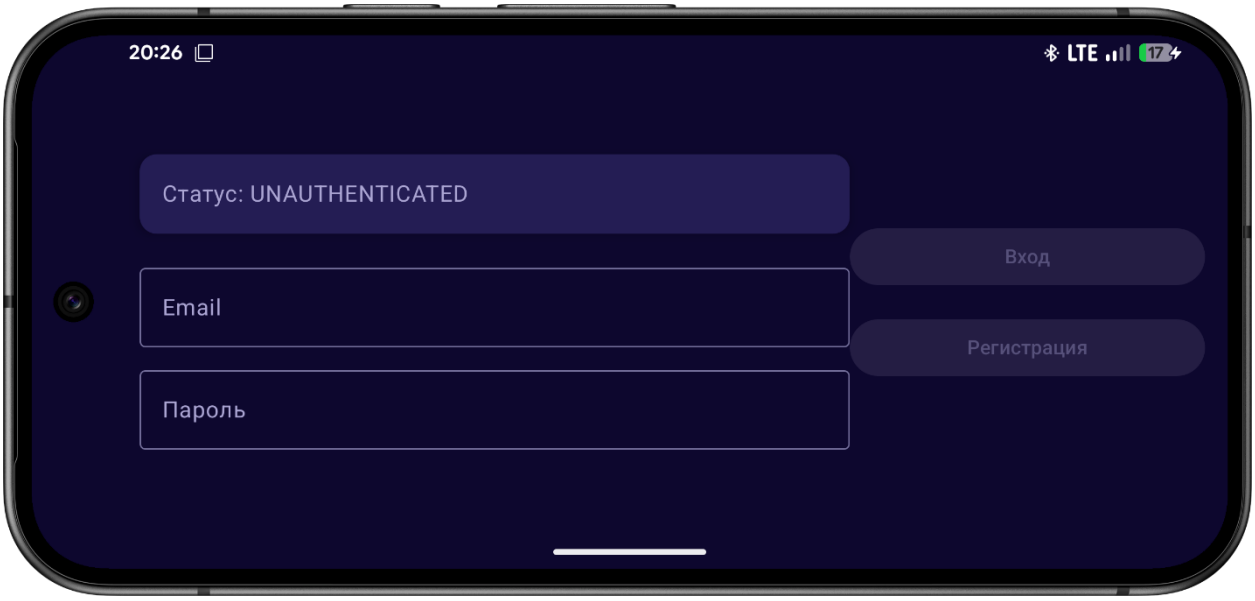

Рисунок 2.1 – Экран авторизации приложения

Состояние пользовательских документов в Firebase Cloud Firestore показано на рисунке 2.2.

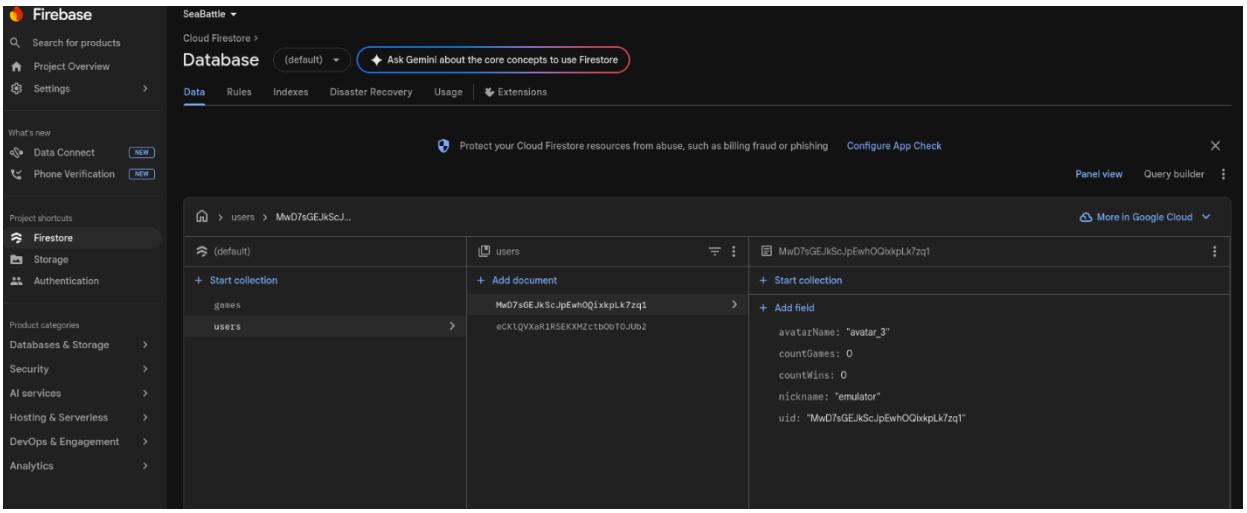

Рисунок 2.2 – Хранение профилей пользователей в Firebase

После успешной авторизации пользователь попадает на экран профиля. В нем реализованы просмотр текущего никнейма, изменение имени, выбор аватара из набора встроенных изображений, а также переход к созданию новой игры, просмотру активных партий и заготовке под экран статистики. Изменение никнейма и имени аватара сохраняется через UserRepository в Firestore, благодаря чему профиль остается доступным после повторного входа в систему. Экран профиля целесообразно привести на рисунке 2.3.

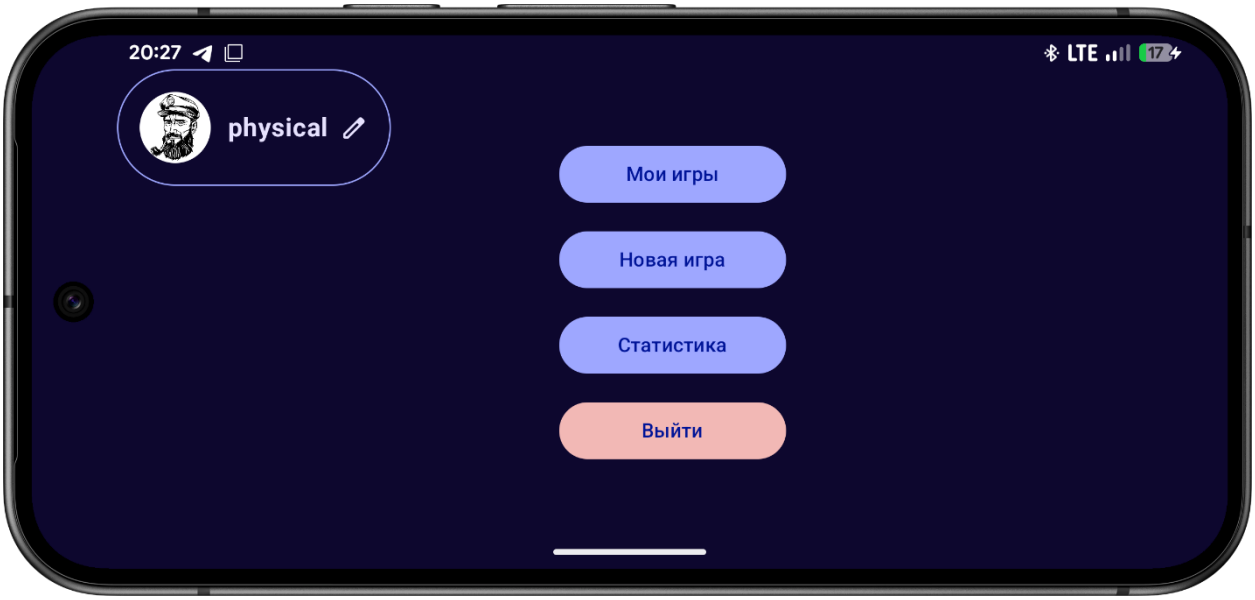

Рисунок 2.3 – Экран профиля пользователя

Создание и подключение к партии выполняются на рисунке 2.4. Первый игрок нажимает кнопку создания игры, после чего в коллекции games создается документ с уникальным gameId, идентификатором хозяина партии hostId, признаком состояния WAITING. Сформированный идентификатор можно скопировать и передать второму игроку. Второй пользователь вводит

полученный ID, а затем подключается к партии транзакционным обновлением записи: в документ игры добавляется guestId, а статус меняется на ACTIVE.

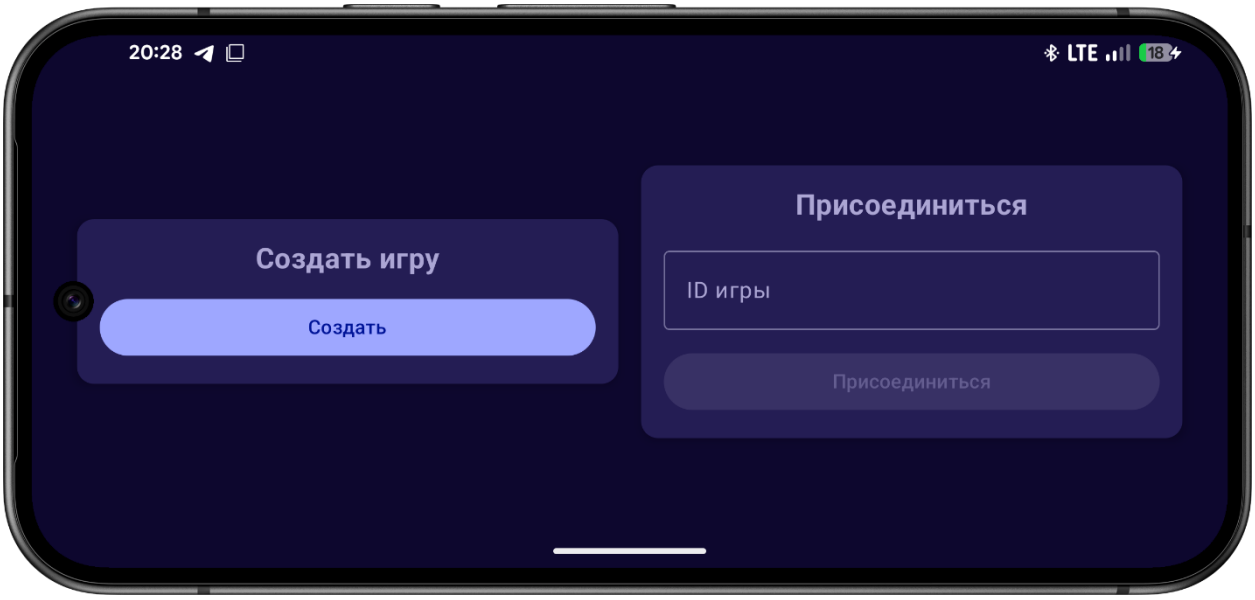

Рисунок 2.4 – Экран создания и подключения к игре

Расстановка флота на поле размером 10x10. Для этого в приложении реализован отдельный экран, на котором пользователь может выбирать тип корабля, менять ориентацию их, размещать корабли вручную, отменять последнее действие, очищать поле и выполнять случайную автоматическую расстановку. В PlacementViewModel заложены проверки выхода за границы, наложения кораблей друг на друга и касания соседних кораблей, что соответствует классическим правилам "Морского боя". Экран расстановки можно продемонстрирован на рисунке 2.5.

Рисунок 2.5 – Экран расстановки флота

Игровой экран содержит два поля: собственное и поле соперника. На поле противника пользователь выбирает клетку для атаки, после чего BattleViewModel инициирует транзакцию makeMove(...) в GameRepository. Внутри транзакции выполняется чтение актуального состояния партии, проверка очередности хода и обновление атакуемого массива клеток. Пустая клетка переводится в состояние MISS, палуба корабля - в HURT, а полностью уничтоженный корабль помечается как DESTROYED. Благодаря addSnapshotListener(...) оба клиента получают обновления мгновенно и синхронно перерисовывают интерфейс. Экран боя показан на рисунке 2.6.

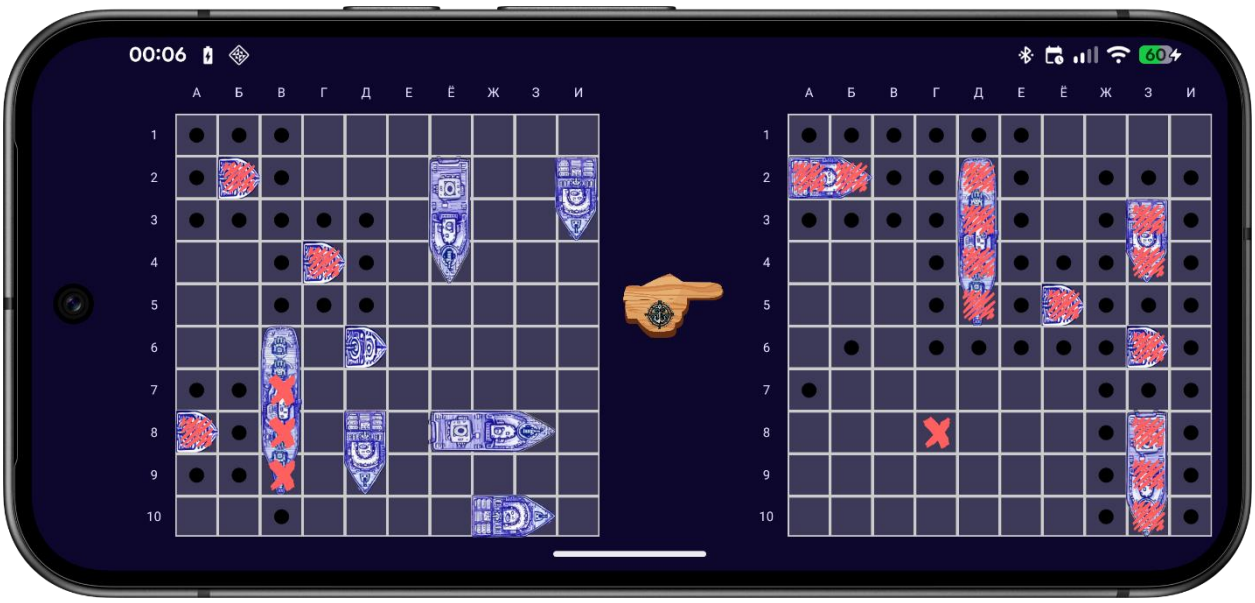

Рисунок 2.6 – Экран игрового процесса

Для более наглядного представления архитектуры и динамики работы системы были подготовлены UML-диаграммы. Диаграмма последовательности, представленная на рисунке 2.7, отражает полный сценарий от авторизации пользователей до создания игры, подключения соперника, подтверждения расстановки и выполнения хода.

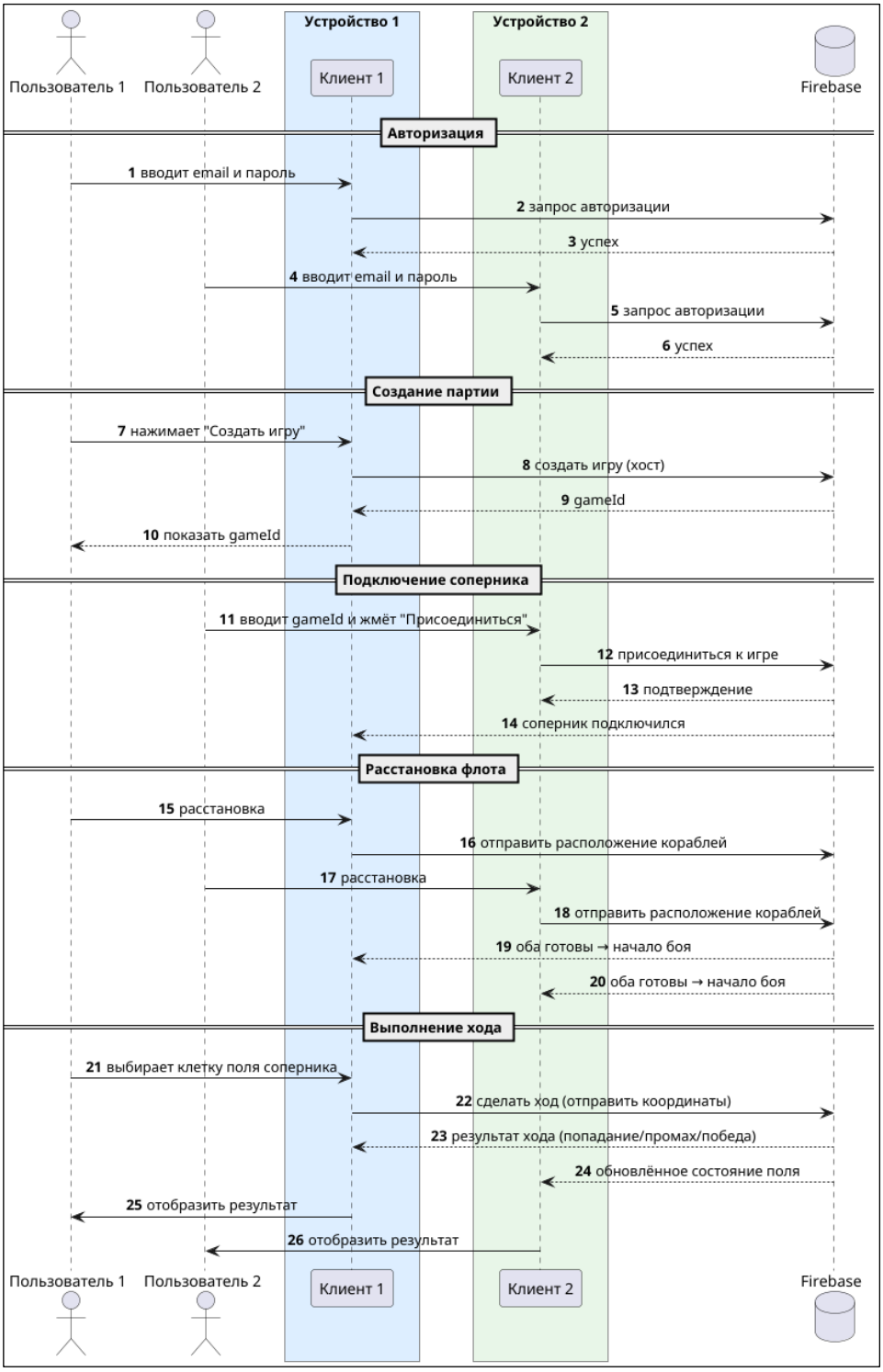

Рисунок 2.7 – Диаграмма последовательности

Диаграмма развертывания, представленная на рисунке 2.8, показывает распределение компонентов приложения между двумя устройствами Android и внешними сервисами Firebase.

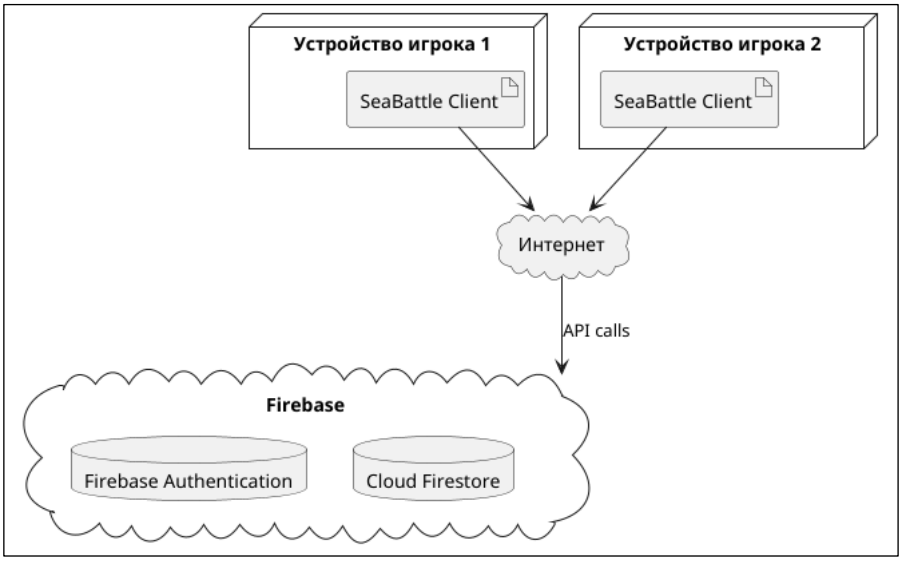

Рисунок 2.8 – Диаграмма развёртывания

Диаграмма состояний, приведенная на рисунке 2.9, описывает жизненный цикл партии: ожидание соперника, подготовку игровых полей, активную фазу боя и завершение игры.

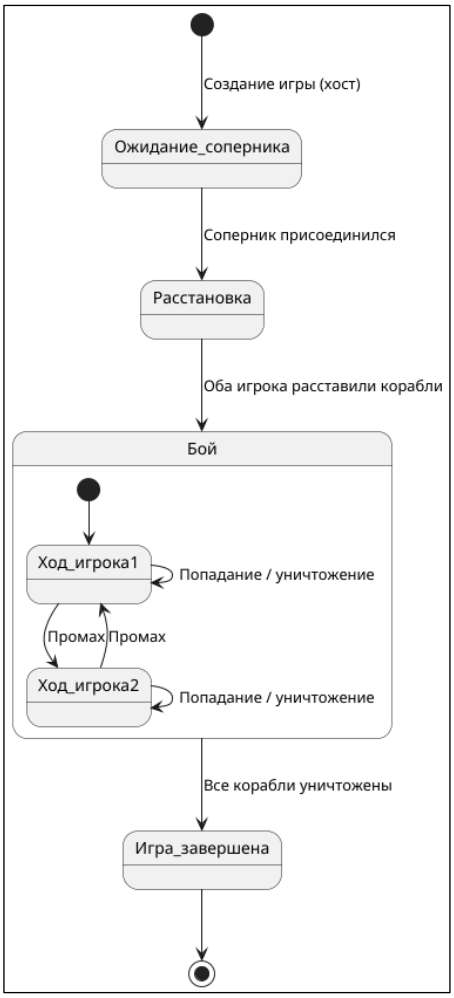

Рисунок 2.9 – Диаграмма состояний

ВЫВОД

В результате выполнения лабораторной работы было разработано клиент-серверное мобильное приложение "Морской бой", обеспечивающее авторизацию игроков, создание и подключение к сетевой партии, расстановку флота, проведение боя и синхронизацию состояния между двумя устройствами через Firebase. Использование Kotlin, Jetpack Compose, Navigation Compose, Firestore и архитектуры MVVM позволило построить понятную структуру проекта с разделением ответственности между экраном, слоем состояния и репозиториями данных.